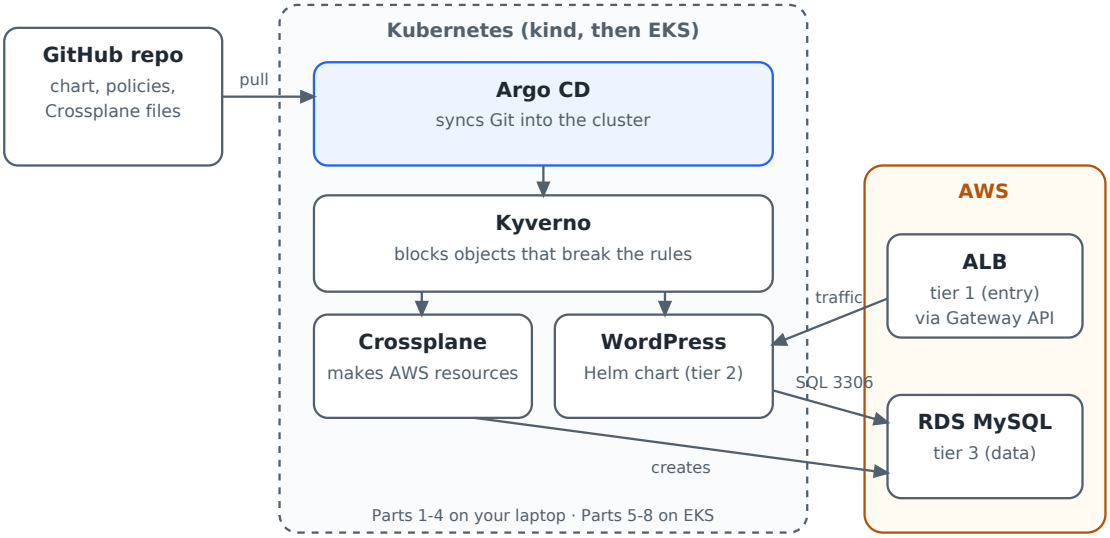
Hands-on lab: WordPress on Kubernetes

Learn Helm, Argo CD, Kyverno and Crossplane by building one three-tier app (load balancer, WordPress, RDS). Written for AWS engineers. Plain steps and no YAML, except for the Gateway API in Part 5. Each step links to the official docs that show the exact files.

As of November 2026 · Crossplane v2.4 · Kyverno 1.20 · EKS Auto Mode



What you build: Argo CD pulls everything from Git, Kyverno checks it, Helm packages WordPress, Crossplane creates the RDS database.

How to use this lab

- Do the parts in order. Each one adds one tool and builds on the last.
- Every part has the same shape: **Goal**, **In AWS terms**, **Steps**, **You are done when**, **Docs**.
- When a step needs a YAML file, open the linked doc, copy its example and adapt it. Writing those files yourself is the skill you are learning.
- Keep everything in one Git repo. By the end it is your portfolio.
- If a step doesn't make sense after an hour, stop and ask. One unclear concept slows everything after it.
- Parts 0-4 run on your laptop for free. Parts 5-8 use real AWS resources that cost money while they run: set a budget alarm first and tear down after each session (Part 9).

The four tools in AWS terms

Tool	What it does	Closest AWS idea
Helm	Packages many Kubernetes files into one versioned, configurable app	A CloudFormation template with Parameters, stack updates and rollback
Argo CD	Watches Git and applies it to the cluster; undoes manual changes	CodePipeline deploying from a repo, plus automatic drift correction
Kyverno	Rejects Kubernetes objects that break your rules	SCPs or AWS Config rules, applied to Kubernetes objects
Crossplane	Creates AWS resources from Kubernetes objects and keeps fixing drift	CloudFormation that never stops running

PART 0

Prepare your laptop

Time: 30 minutes · Where: your laptop

Goal
Have every tool installed and an empty Git repo ready.

In AWS terms
Like setting up your AWS CLI profile and a sandbox account before starting a project.

Steps

1. Install Docker (Docker Desktop on Mac/Windows). kind runs Kubernetes inside Docker containers.
2. Install `kubectl`, the command you use to talk to any Kubernetes cluster.
3. Install `kind` (Kubernetes in Docker) for the free local cluster.
4. Install `helm` version 3 or newer.
5. Install the `argocd` CLI.
6. Optional: install the `kyverno` CLI to test policies without a cluster.
7. Check that `aws` CLI v2 and `eksctl` work (you need them from Part 5).
8. Create an empty GitHub repository called `wordpress-lab` with four folders: `charts/`, `argocd/`, `policies/`, `infra/`.

You are done when

- `kubectl version --client`, `kind version`, `helm version` and `argocd version --client` all print a version.
- Your empty repo is cloned on your laptop.

Docs that help with these steps

- Get Docker
  https://docs.docker.com/get-started/get-docker/
- Install kubectl
  https://kubernetes.io/docs/tasks/tools/
- kind Quick Start (install + create cluster)
  https://kind.sigs.k8s.io/docs/user/quick-start/
- Installing Helm
  https://helm.sh/docs/intro/install/
- Argo CD CLI installation
  https://argo-cd.readthedocs.io/en/stable/cli_installation/
- Kyverno CLI
  https://kyverno.io/docs/subprojects/kyverno-cli/
- Set up to use Amazon EKS (AWS CLI, eksctl)
  https://docs.aws.amazon.com/eks/latest/userguide/setting-up.html

PART 1

The three tiers by hand (no tools yet)

Time: 1-2 hours · Where: your laptop (kind)

Goal
Run WordPress and MySQL on a local cluster with plain kubectl, so you know exactly what Helm will package in Part 2.

In AWS terms
Like building a stack once by hand in the console before you write the CloudFormation template.

Steps

1. Create a local cluster: `kind create cluster --name wp-lab`.
2. Follow the official Kubernetes tutorial **Deploying WordPress and MySQL with Persistent Volumes** step by step. It gives you all the files.
3. Name the pieces you created: a **Secret** (DB password), **MySQL** Deployment + PersistentVolumeClaim (tier 3), **WordPress** Deployment (tier 2), and a **Service** in front of WordPress (tier 1 for now).
4. Open the site with `kubectl port-forward` to the WordPress Service and finish the WordPress install wizard. Write one blog post.
5. Break it on purpose: delete the MySQL Pod with `kubectl delete pod`. Watch it come back with `kubectl get pods -w`. Your post is still there because the data lives on the volume, not in the Pod.
6. Clean up the way the tutorial shows, but keep the cluster for Part 2.

You are done when

- You can explain in one sentence what each of the five objects does.
- You have used `get`, `describe`, `logs` and `port-forward` at least once.

Docs that help with these steps

- Tutorial: Deploying WordPress and MySQL with Persistent Volumes
  https://kubernetes.io/docs/tutorials/stateful-application/mysql-wordpress-persistent-volume/
- Deployments
  https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- Services
  https://kubernetes.io/docs/concepts/services-networking/service/
- Secrets
  https://kubernetes.io/docs/concepts/configuration/secret/
- Persistent Volumes
  https://kubernetes.io/docs/concepts/storage/persistent-volumes/
- Use port forwarding to access applications
  https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/

PART 2

Helm: package the stack

Time: 2-3 hours · Where: your laptop (kind)

Goal
Turn the files from Part 1 into one Helm chart that installs, upgrades and rolls back the whole app with one command.

In AWS terms
A CloudFormation template with Parameters: values.yaml is the parameter file, a Helm *release* is the stack, and helm rollback is a stack rollback.

Watch out
Don't use the Bitnami WordPress chart from older tutorials. Bitnami ended its free images and charts in 2025, so those installs now fail or get no security updates. Building your own chart is the better way to learn Helm anyway.

Steps

1. Read the Helm Quickstart. Then, inside your repo, run `helm create wordpress` in the `charts/` folder.
2. Look at what `helm create` generated (Chart.yaml, values.yaml, templates/). Then delete the example files in templates/ and copy in your files from Part 1, one object per file.
3. Make one value configurable at a time: first the WordPress image tag. Put it in values.yaml, reference it in the template, and run `helm template` to see the result. Then do replicas, storage size and the database name.
4. Add two switches for later: `mysql.enabled` (true/false around the MySQL files) and `externalDatabase.host`. In Part 7 you turn the in-cluster MySQL off and point WordPress to RDS. The *Flow Control* doc shows the if-block.
5. Run `helm lint` and `helm template` until both are clean.
6. Install: `helm install blog ./charts/wordpress -n team-blog --create-namespace`. Open the site with port-forward.
7. Upgrade and roll back: `helm upgrade` with a different image tag, check `helm history blog -n team-blog`, then `helm rollback blog 1 -n team-blog`.
8. Uninstall with `helm uninstall` and commit the chart to Git. From now on Argo CD installs it, not you.

You are done when

- One command installs all the tiers, and you can roll back.
- You can point to the line in values.yaml that controls each setting.

Docs that help with these steps

- Helm Quickstart Guide
  https://helm.sh/docs/intro/quickstart/
- Using Helm (install, upgrade, rollback, create)
  https://helm.sh/docs/intro/using_helm/
- Chart Template Guide: Getting Started
  https://helm.sh/docs/chart_template_guide/getting_started/
- Values Files
  https://helm.sh/docs/chart_template_guide/values_files/
- Flow Control (if / else)
  https://helm.sh/docs/chart_template_guide/control_structures/
- helm template (render without installing)
  https://helm.sh/docs/helm/helm_template/
- Chart Best Practices
  https://helm.sh/docs/chart_best_practices/

PART 3

Argo CD: deploy from Git

Time: 2 hours · Where: your laptop (kind)

Goal
Stop running helm install yourself. Argo CD installs your chart from Git and keeps the cluster equal to Git.

In AWS terms
CodePipeline watching a repo, plus automatic drift correction: if someone changes something by hand, Argo CD puts it back.

Steps

1. Install Argo CD into the `argocd` namespace, using the Getting Started guide or the official Helm chart.
2. Get the initial admin password, port-forward the UI, and log in with the browser and with `argocd login`.
3. Push your repo to GitHub. A public repo needs no setup; for a private repo, add the repo credentials (Private Repositories doc).
4. Create an **Application** that points to `charts/wordpress` with destination namespace `team-blog`. Do it once in the UI to see the fields, then again as a file in `argocd/` (Declarative Setup doc). From now on, always use the file.
5. Turn on automated sync with **prune** and **self-heal**.
6. Test GitOps: change replicas in values.yaml, push, and watch Argo CD roll it out with no command from you.
7. Test drift: `kubectl scale` the WordPress Deployment by hand. Self-heal changes it back. Then delete the Service: it comes back too.
8. Create an **AppProject** called `team-blog` that only allows the `team-blog` namespace, and move your Application into it. Try pointing the Application at another namespace: Argo CD refuses.

You are done when

- Git is the only way you change the app.
- You have seen self-heal undo a manual change.
- You can explain the difference between an Application and an AppProject.

Docs that help with these steps

- Argo CD Getting Started
  https://argo-cd.readthedocs.io/en/stable/getting_started/
- Argo CD official Helm chart
  https://github.com/argoproj/argo-helm
- Declarative Setup (Applications and Projects as files)
  https://argo-cd.readthedocs.io/en/stable/operator-manual/declarative-setup/
- Helm charts in Argo CD
  https://argo-cd.readthedocs.io/en/stable/user-guide/helm/
- Automated Sync Policy (prune, self-heal)
  https://argo-cd.readthedocs.io/en/stable/user-guide/auto_sync/
- Projects
  https://argo-cd.readthedocs.io/en/stable/user-guide/projects/
- Private Repositories
  https://argo-cd.readthedocs.io/en/stable/user-guide/private-repositories/
- Application Specification Reference (every field explained)
  https://argo-cd.readthedocs.io/en/stable/user-guide/application-specification/

PART 4

Kyverno: guardrails

Time: 2 hours · Where: your laptop (kind)

Goal
Block bad configurations before they reach the cluster, and see the result in Argo CD.

In AWS terms
SCPs or AWS Config rules, but for Kubernetes objects: a rule says what is not allowed, and the request is denied with your message.

Watch out
Use the **ValidatingPolicy** kind (CEL expressions). Many blog posts still show the older **ClusterPolicy** kind. It is deprecated, and Kyverno 1.20 no longer enforces those legacy policies. If a tutorial shows ClusterPolicy, use the Quick Start instead.

Steps

1. Start in the browser: try the example policies in the **Kyverno Playground**. No cluster needed.
2. Install Kyverno **through Argo CD**: add a second Application that installs the `kyverno` chart from the Kyverno Helm repository. This also shows that Argo CD can install charts from a Helm repo, not only from Git.
3. Write three **ValidatingPolicy** files in `policies/` and add a third Application for that folder: (a) every Pod needs a `team` label, (b) no `:latest` image tags, (c) every container sets CPU and memory requests.
4. Start with the **Audit** action. Run `kubectl get policyreport -A` to see what would fail.
5. Fix your WordPress chart until the reports are clean, then push.
6. Switch the policies to **Deny**. Try `kubectl run test --image=nginx:latest`: it is rejected with your message.
7. Optional: install the Pod Security Standards policy set with the kyverno-policies chart.

You are done when

- A bad change is blocked, and you can read the error and fix it.
- Your WordPress chart passes all three policies.

Docs that help with these steps

- Kyverno Playground (try policies in the browser)
  https://playground.kyverno.io/
- Kyverno Quick Start (uses ValidatingPolicy)
  https://kyverno.io/docs/introduction/quick-start/
- Installation (Helm)
  https://kyverno.io/docs/installation/installation/
- Kyverno CLI (test policies offline)
  https://kyverno.io/docs/subprojects/kyverno-cli/
- Pod Security Standards policies
  https://kyverno.io/docs/guides/pod-security/
- Policy library (examples)
  https://kyverno.io/policies/
- Upgrading notes (why old ClusterPolicy files stop working)
  https://kyverno.io/docs/installation/upgrading/

PART 5

Move to AWS: EKS Auto Mode + Gateway API

Time: 2 hours · Where: AWS (costs money while running)

Goal
Run the same Git repo on a real EKS cluster, with a real Application Load Balancer as tier 1, created through the Kubernetes Gateway API.

In AWS terms
Promoting from a dev sandbox to a real account: same templates, new target. In the Gateway API, **GatewayClass** is like choosing "ALB" plus account-wide defaults, **Gateway** is the ALB and its listener, and **HTTPRoute** is the listener rule that forwards to a target group.

Watch out
Many tutorials still use **Ingress**. Ingress is not removed, but it is frozen (no new features), and Gateway API is its successor, so use the files on the next page instead. Also keep WordPress at 1 replica on EKS for now: its volume attaches to one node only.

Steps

1. Create an **AWS Budget** with an email alert before anything else.
2. Create an EKS Auto Mode cluster with one command: `eksctl create cluster --name wp-lab --region eu-central-1 --enable-auto-mode`. Auto Mode already includes nodes, the load balancer controller and Pod Identity.
3. Turn on the Gateway API, so AWS installs its CRDs for you: `aws eks update-cluster-config --name wp-lab --kubernetes-network-config '{"elasticLoadBalancing":{"enabled":true,"gatewayApi":{"crds":{"managed":true}}}}'`. Wait until `kubectl get crd gateways.gateway.networking.k8s.io` works.
4. Put the **GatewayClass** file (next page) in Git, in a platform folder that Argo CD applies. This is the platform team's part.
5. Add the **Gateway + HTTPRoute** template (next page) to your Helm chart, switched on by `gateway.enabled: true` in values. This is the app team's part.
6. Install Argo CD on EKS the same way as in Part 3, then apply your Application files. Kyverno and your policies come along automatically, because they are in Git.
7. Wait for the address in `kubectl get gateway -n team-blog` (2-3 minutes) and open `http://ADDRESS` in the browser.
8. WordPress still uses the MySQL inside the cluster. Part 7 changes that.

You are done when

- WordPress opens through the ALB address shown on the Gateway.
- You can say which object made the ALB (Gateway), which made the rule (HTTPRoute), and who owns each.
- You did not run kubectl apply for the app: everything came from Git.

Docs that help with these steps

- Configure the Kubernetes Gateway API in EKS Auto Mode
  https://docs.aws.amazon.com/eks/latest/userguide/auto-configure-gateway-api.html
- Gateway API: concepts and API overview
  https://gateway-api.sigs.k8s.io/concepts/api-overview/
- Gateway API: HTTP routing guide
  https://gateway-api.sigs.k8s.io/guides/http-routing/
- LoadBalancerConfiguration options (scheme, subnets, ...)
  https://kubernetes-sigs.github.io/aws-load-balancer-controller/latest/guide/gateway/customization/
- Creating an AWS budget
  https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-create.html

Gateway API code

1. Platform team: GatewayClass (once per cluster)

```
# platform/gatewayclass.yaml  (owned by the platform team, applied by Argo CD)
apiVersion: v1
kind: Namespace
metadata:
  name: platform-gateway
---
# The only AWS-specific object: the ALB is internal by default, this makes it public.
apiVersion: gateway.k8s.aws/v1
kind: LoadBalancerConfiguration
metadata:
  name: internet-facing
  namespace: platform-gateway
spec:
  scheme: internet-facing
---
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: eks-auto-alb
spec:
  controllerName: eks.amazonaws.com/gateway-alb   # ALB, managed by Auto Mode
  parametersRef:                                  # every Gateway of this class is public
    group: gateway.k8s.aws
    kind: LoadBalancerConfiguration
    name: internet-facing
    namespace: platform-gateway
```

2. App team: Gateway + HTTPRoute (in your Helm chart)

```
# charts/wordpress/templates/gateway.yaml  (part of your Helm chart)
{{- if .Values.gateway.enabled }}
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway                     # = one ALB with one listener
metadata:
  name: {{ .Release.Name }}
spec:
  gatewayClassName: eks-auto-alb
  listeners:
    - name: http
      protocol: HTTP
      port: 80
      allowedRoutes:
        namespaces:
          from: Same              # only routes from this namespace may attach
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute                   # = the listener rule
metadata:
  name: {{ .Release.Name }}
spec:
  parentRefs:
    - name: {{ .Release.Name }}
      sectionName: http
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: {{ include "wordpress.fullname" . }}   # your WordPress Service
          port: 80
{{- end }}
```

3. Check it

```
kubectl get gatewayclass eks-auto-alb           # ACCEPTED=True
kubectl get gateway,httproute -n team-blog      # PROGRAMMED=True, ADDRESS filled
kubectl describe httproute -n team-blog         # Status > Parents > Accepted, ResolvedRefs
```

PART 6

Crossplane basics: one bucket

Time: 2 hours · Where: AWS

Goal
Create an AWS resource from a Kubernetes object, and watch it heal itself.

In AWS terms
CloudFormation that never stops running: if someone deletes the bucket in the console, Crossplane creates it again within minutes.

Watch out
The service account name must match in two places: the Pod Identity association and the DeploymentRuntimeConfig. If they differ, the provider reports missing credentials. This is the most common mistake in this part.

Steps

1. Install Crossplane with its Helm chart **through Argo CD** into the `crossplane-system` namespace.
2. Read **Get Started With Managed Resources**. It uses access keys; you will use EKS Pod Identity instead, so there are no keys anywhere.
3. Create an IAM role for Crossplane and a Pod Identity association for namespace `crossplane-system` and service account `provider-aws`. `eksctl create podidentityassociation` can do both in one command.
4. Give the provider that fixed service account name with a **DeploymentRuntimeConfig**, then install `provider-aws-s3` pointing to it (Upbound Pod Identity guide).
5. Create a **ClusterProviderConfig** with credentials source `PodIdentity`.
6. Add a **Bucket** managed resource to `infra/`. Argo CD applies it. Check that `kubectl get buckets.s3.aws.m.upbound.io -A` shows SYNCED=True and READY=True.
7. Delete the bucket in the S3 console. Watch Crossplane create it again.

You are done when

- You have seen Crossplane recreate a deleted AWS resource.
- You can explain Provider, ProviderConfig and managed resource in your own words.

Docs that help with these steps

- Install Crossplane
  https://docs.crossplane.io/latest/get-started/install/
- Get Started With Managed Resources
  https://docs.crossplane.io/latest/get-started/get-started-with-managed-resources/
- Upbound: AWS authentication with Pod Identity
  https://docs.upbound.io/manuals/packages/providers/aws-auth/aws-pod-identity/
- EKS Pod Identity
  https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html
- Crossplane Providers (incl. DeploymentRuntimeConfig)
  https://docs.crossplane.io/latest/packages/providers/

PART 7

Crossplane: RDS as the data tier

Time: 2-3 hours · Where: AWS

Goal
Replace the in-cluster MySQL with an RDS database that Crossplane creates.

In AWS terms
The same four pieces you would put in a CloudFormation RDS stack: a security group, an ingress rule, a DB subnet group and the DB instance.

Steps

1. Install `provider-aws-rds` and `provider-aws-ec2` with the same DeploymentRuntimeConfig. Add RDS and security-group permissions to Crossplane's IAM role.
2. Look up the cluster's VPC ID, cluster security group and private subnet IDs (`aws eks describe-cluster`, `aws ec2 describe-subnets`).
3. In `infra/team-blog/`, create four managed resources: (a) a **SecurityGroup** in the cluster VPC, (b) a **SecurityGroupRule** allowing TCP 3306 only from the cluster security group, (c) a **SubnetGroup** with the private subnets, (d) an **Instance**: MySQL 8.0, db.t4g.micro, 20 GB, encrypted, not public, with a generated password written to a Secret in `team-blog`.
4. Push. Wait about 10 minutes, then read the endpoint from the Instance status.
5. In your chart's values: set `mysql.enabled=false`, set `externalDatabase.host` to the endpoint, and read the password from the Secret Crossplane created. Push; Argo CD redeploys WordPress.
6. Kyverno bonus: write a ValidatingPolicy that denies any RDS Instance with `publiclyAccessible: true`. Kyverno checks Crossplane objects too, just like an SCP would.

You are done when

- WordPress runs on RDS, and the database is not reachable from the internet.
- Your Kyverno policy blocks a public RDS instance.

Docs that help with these steps

- provider-aws-rds (all RDS resources and fields)
  https://marketplace.upbound.io/providers/upbound/provider-aws-rds
- provider-aws-ec2 (SecurityGroup, SecurityGroupRule)
  https://marketplace.upbound.io/providers/upbound/provider-aws-ec2
- Get Started With Managed Resources (pattern to copy)
  https://docs.crossplane.io/latest/get-started/get-started-with-managed-resources/
- Amazon RDS for MySQL
  https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_MySQL.html
- RDS in a VPC (DB subnet groups)
  https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_VPC.WorkingWithRDSInstanceinaVPC.html
- Controlling access with security groups (RDS)
  https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Overview.RDSSecurityGroups.html

PART 8

Crossplane composition: your own database API

Time: 2-3 hours · Where: AWS

Goal
Package the four RDS resources into one simple object that a team can request in a few lines.

In AWS terms
A Service Catalog product: the team picks small or medium, and the platform decides the instance class, network and encryption.

Steps

1. Do **Get Started With Composition** and choose the plain YAML (patch-and-transform) option. Skip the other languages for now.
2. Design the API on paper first: kind `WordPressDatabase`, one input field `size` (small or medium), one output field `endpoint`.
3. Write the **XRD** (namespaced) for that design.
4. Install `function-patch-and-transform` and `function-environment-configs`. Put the VPC, subnet and security group IDs into an **EnvironmentConfig**.
5. Write the **Composition** using the four resources from Part 7 as bases. Patches: map `size` to the instance class (small to db.t4g.micro, medium to db.t4g.medium), read the network IDs from the environment, and copy the endpoint back to the XR status.
6. Remove Part 7's individual resources from Git, and add one `WordPressDatabase` in `team-blog` instead.
7. Lock it down: let the `team-blog` AppProject create only WordPressDatabase plus the app's own kinds. Your Kyverno policy from Part 7 still protects RDS.

You are done when

- A team gets a working database by writing about six lines.
- Changing size from small to medium resizes the database.

Docs that help with these steps

- Get Started With Composition
  https://docs.crossplane.io/latest/get-started/get-started-with-composition/
- Composite Resource Definitions (XRD)
  https://docs.crossplane.io/latest/composition/composite-resource-definitions/
- Compositions
  https://docs.crossplane.io/latest/composition/compositions/
- Function Patch and Transform (patches, map transform)
  https://docs.crossplane.io/latest/guides/function-patch-and-transform/
- Environment Configs
  https://docs.crossplane.io/latest/composition/environment-configs/
- Connection Details Composition (passing the DB secret)
  https://docs.crossplane.io/latest/guides/connection-details-composition/

PART 9

Clean up (the order matters)

> **Why the order matters**
> If you delete the EKS cluster first, Crossplane disappears with it and nothing deletes the RDS database or the ALB. They keep running and keep costing money.

Steps

1. Remove the WordPressDatabase (or the RDS resources) from Git and wait until the database is gone in the RDS console.
2. Remove the app (its Gateway and HTTPRoute), and check that the ALB is gone in the EC2 console. Then remove the GatewayClass.
3. Delete the cluster: `eksctl delete cluster --name wp-lab --region eu-central-1`.
4. Delete Crossplane's IAM role. Look for leftover security groups and EBS volumes.
5. Delete the local cluster: `kind delete cluster --name wp-lab`.

Appendix A: debugging cheat sheet

When...	Run this
A Pod is not running	`kubectl describe pod NAME -n NS` then `kubectl logs NAME -n NS` and `kubectl get events -n NS --sort-by=.lastTimestamp`
Helm output looks wrong	`helm template blog ./charts/wordpress` (render without installing), `helm get manifest blog -n NS`
Argo CD shows OutOfSync or Degraded	`argocd app get APP` and `argocd app diff APP`, or open the app in the UI and click the red object
Kyverno blocked something	Read the message in the error; then `kubectl get policyreport -A`
A Crossplane resource is not READY	`kubectl describe` it and read the Events at the bottom; `crossplane beta trace KIND NAME -n NS` for compositions; `kubectl get managed`
The Gateway has no ADDRESS	`kubectl describe gateway NAME -n NS`: read Conditions (Accepted, Programmed)
Image pull fails with an x509 error	You are probably behind a company TLS proxy: load the image into kind with `kind load docker-image`

Appendix B: progress checklist

Part	Milestone	Done
0	All tools installed, repo created	☐
1	WordPress + MySQL running by hand on kind	☐
2	Own Helm chart installs, upgrades and rolls back	☐
3	Argo CD deploys from Git; self-heal seen	☐
4	Three Kyverno policies enforced	☐
5	WordPress on EKS behind an ALB made by the Gateway API	☐
6	Crossplane bucket recreated after deletion	☐
7	WordPress on RDS created by Crossplane	☐
8	WordPressDatabase API with small / medium	☐
9	Everything deleted, no leftover cost	☐